

AI/ML intern DAY 7

TASK — Body Position and Angle Detection System

LOYA PRANAY (pranay8679@gmail.com)

Date: 19-05-2026

1. Introduction

This project focuses on developing an AI-Based Body Position and Angle Detection System using Artificial Intelligence, Computer Vision, MediaPipe, and OpenCV.

The system analyses uploaded human body images and performs:

- body landmark detection
- skeletal visualization
- body angle estimation
- body posture analysis
- body orientation tracking

The AI system detects human body posture and analyses body alignment using pose estimation techniques.

2. Objective

The objective of this project was to:

- detect human body landmarks
 - analyse body posture
 - calculate body orientation angles
 - detect body position
 - improve AI skeletal tracking systems
 - study pose estimation workflows
-

3. Technologies Used

Technology	Purpose
Python	Programming
OpenCV	Image Processing
MediaPipe	Human Pose Detection

Artificial Intelligence	Pose Analysis
Computer Vision	Body Tracking

4. Features Implemented

The AI system successfully implemented:

- human landmark detection
 - skeletal tracking
 - body angle detection
 - body position estimation
 - posture analysis
 - AI pose visualization
-

5. Workflow of the System

Step 1 — Human Image Input

The AI system accepts uploaded human body images for posture analysis.

Step 2 — Human Landmark Detection

MediaPipe Pose Estimation detects:

- shoulders
- elbows
- wrists
- hips
- knees
- ankles

using AI body tracking.

Step 3 — Coordinate Extraction

The AI system extracts:

- x coordinates
- y coordinates

from body landmarks.

Step 4 — Angle Calculation

The body angle is calculated using landmark coordinate analysis.

The angle calculation helps analyse:

- body tilt
- shoulder alignment
- body orientation
- posture direction

The body angle is calculated using:

$$\theta = \tan^{-1} \left(\frac{y_2 - y_1}{x_2 - x_1} \right)$$

-10-8-6-4-2246810-10-5510A(-8, -8)B(8, 8)m = 1.00

Step 5 — Position Detection

The AI system detects:

- standing straight posture
- tilted posture
- body orientation
- posture alignment

using skeletal tracking.

6. Code Implementation:

```
1  import cv2
2  import mediapipe as mp
3  import math
4
5  image = cv2.imread("human.jpg")
6
7  if image is None:
8      print("Image not found")
9      exit()
10
11  image = cv2.resize(image, (720, 720))
12
13  mp_pose = mp.solutions.pose
14  mp_draw = mp.solutions.drawing_utils
15
16  pose = mp_pose.Pose(
17      static_image_mode=True,
18      min_detection_confidence=0.7
19  )
20
21  rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
22
23  results = pose.process(rgb)
24
25  if results.pose_landmarks:
```

```
26
27      lm = results.pose_landmarks.landmark
28
29      h, w, _ = image.shape
30
31      ls = lm[11]
32      rs = lm[12]
33
34      x1, y1 = int(ls.x*w), int(ls.y*h)
35      x2, y2 = int(rs.x*w), int(rs.y*h)
36
37      angle = int(
38          math.degrees(
39              math.atan2(y2-y1, x2-x1)
40          )
41      )
42
43      if abs(y1-y2) < 20:
44          position = "Standing Straight"
45
46      elif y1 > y2:
47          position = "Tilted Right"
48
49      else:
50          position = "Tilted Left"
```

```

51
52     mp_draw.draw_landmarks(
53         image,
54         results.pose_landmarks,
55         mp_pose.POSE_CONNECTIONS
56     )
57
58     cv2.putText(
59         image,
60         f"Angle: {angle}",
61         (20,40),
62         cv2.FONT_HERSHEY_SIMPLEX,
63         0.8,
64         (0,255,255),
65         2
66     )
67
68     cv2.putText(
69         image,
70         f"Position: {position}",
71         (20,90),
72         cv2.FONT_HERSHEY_SIMPLEX,
73         0.8,
74         (0,255,0),
75         2
76     )

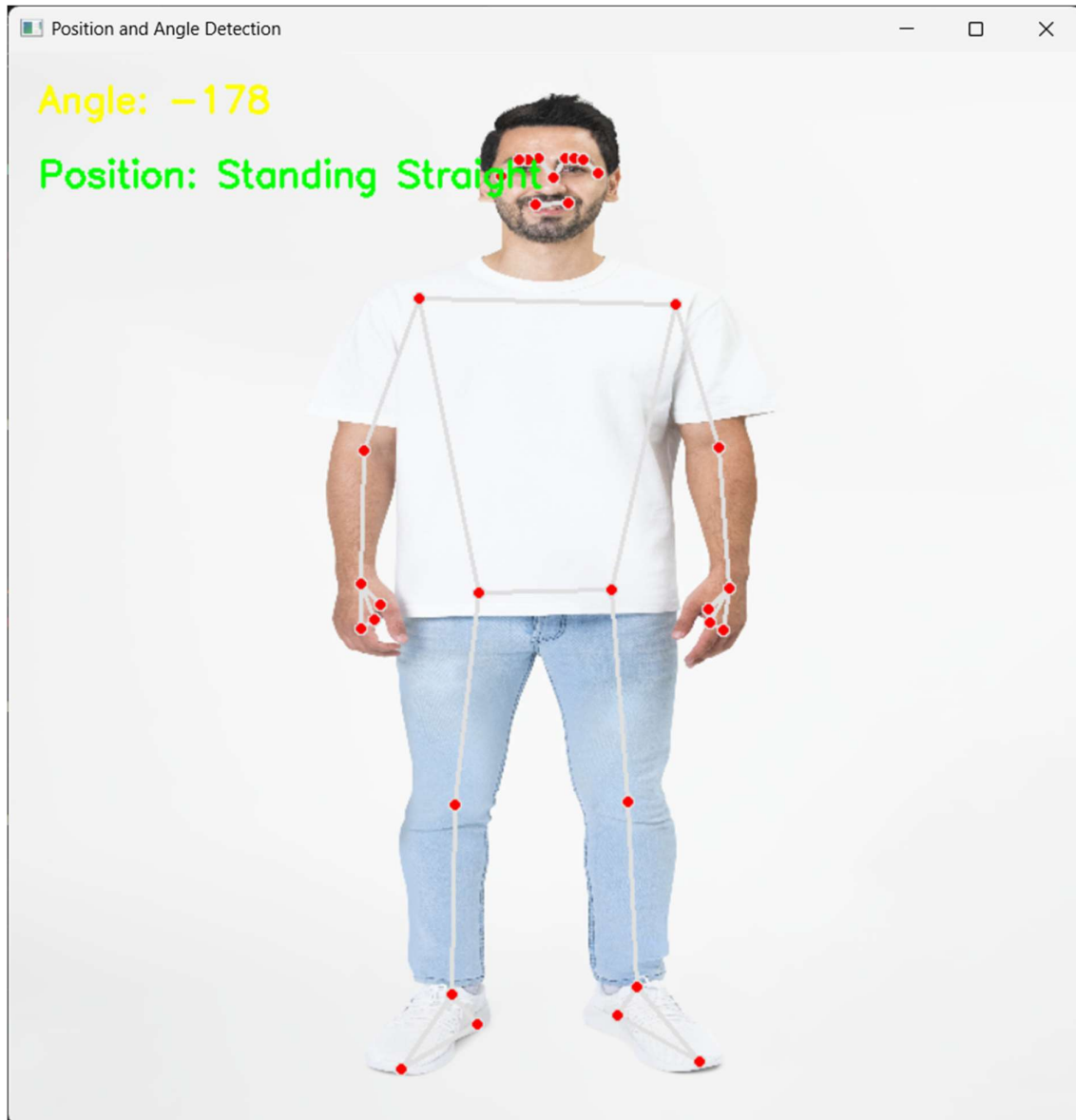
```

```

75         2
76     )
77
78     print("Angle:", angle)
79     print("Position:", position)
80
81 else:
82     print("No Human Detected")
83
84 cv2.imwrite("output.jpg", image)
85
86 cv2.imshow("Position and Angle Detection", image)
87
88 cv2.waitKey(0)
89
90 cv2.destroyAllWindows()

```

Output:



7. Algorithm

Step 1

Load uploaded image using OpenCV.

Step 2

Detect human body landmarks using MediaPipe Pose.

Step 3

Extract body landmark coordinates.

Step 4

Calculate body orientation angle using shoulder coordinates.

Step 5

Analyse body posture and body position.

Step 6

Generate skeletal visualization and posture analysis.

8. Pseudo Code

START

Load uploaded human image

Detect body landmarks

Extract body coordinates

Calculate body angle

Detect body position

Generate skeletal visualization

Display final posture analysis

END

9. Applications

The AI system can be used in:

- fitness monitoring
 - posture correction systems
 - sports analysis
 - rehabilitation systems
 - AI motion tracking
 - healthcare applications
-

10. Advantages

The advantages include:

- accurate posture analysis
 - AI body angle estimation
 - skeletal visualization
 - intelligent body tracking
 - real-time pose estimation
-

11. Challenges Faced

Challenge	Impact
Poor lighting	Reduces landmark accuracy
Side body posture	Affects angle detection
Fast movement	Reduces tracking stability
Complex background	Reduces pose accuracy

12. Expected Output

The AI system generates:

- body landmark visualization
 - skeletal tracking
 - body angle estimation
 - body posture analysis
 - body position detection
-

13. Learning Outcomes

During this project, the following concepts were understood:

- AI pose estimation
- skeletal visualization
- body posture analysis
- body angle calculation
- computer vision workflows

14. Conclusion

The AI-Based Body Position and Angle Detection System successfully demonstrated:

- AI human pose estimation
- body posture analysis
- skeletal tracking
- body angle detection
- intelligent body orientation analysis

using Artificial Intelligence and Computer Vision technologies.

The project improved understanding of:

- body landmark detection
- posture estimation systems
- skeletal visualization
- AI body tracking
- computer vision applications